

DATA301 Project Final Report

Sam Ladbrook

Locality Sensitive Hashing for scalable Collaborative filtering: A Runtime and Accuracy Evaluation on Amazon Movie Reviews

Abstract

This project investigates whether Locality Sensitive Hashing can accelerate item-based collaborative filtering for movie recommendations without sacrificing recommendation accuracy. Using the Amazon Movies and Television Reviews dataset from the McAuley Lab at the University of California San Diego; a sparse, real-world dataset containing millions of user ratings. From this dataset a pipeline was implemented in Python using Dask to compute MinHash hashes, apply Locality Sensitive Hashing banding to identify candidate items and generate top ten recommendations using cosine similarity weighted collaborative filtering. The research question asks how much of a difference exists in both recommendation accuracy and algorithm runtime between the Locality Sensitive Hashing accelerated method and a standard brute force collaborative filtering method. Results show that the accelerated pipeline achieved up to a 2,176 times decrease in runtime over the brute force similarity computation (extrapolated). This reduced an estimated 430 minute runtime to under 12 seconds, while maintaining and sometimes exceeding brute-force recommendation accuracy on identical data subset. These findings are significant for subscription streaming platforms such as Neon, which face commercial pressure to deliver accurate recommendations as larger service but without the equivalent engineering resources.

Introduction

Background

The Amazon Movies and Television Review dataset (McAuley, 2023), contains millions of user ratings for movies and television shows spanning May 1996 to September 2018. Each record includes a user identifier, an item identifier and a star rating on a scale of one to five. I use a five-core subset of this data in this project to ensure sufficient density for meaningful similarity computation. Every user and item that has at least 5 interactions are included in the five-core subset. This five-core subset filtering resulted in 40,946 unique items and over one million ratings. The dataset exhibits a classic long-tail distribution common to e-commerce platforms, where a relatively small number of popular titles attract the majority of ratings while most items are rated by relatively few users. This distribution is clear in Figure 1 below; where ratings per item was plotted.

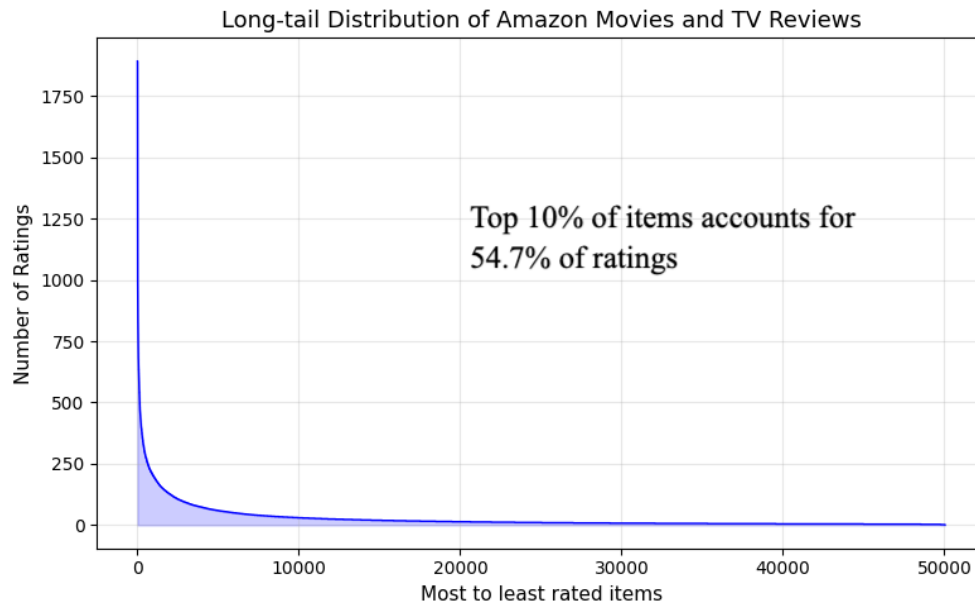


Figure 1: Long tail distribution of ratings per item in the Amazon Movies and TV dataset.

This sparsity directly influences interpretation of the results.

Item-based collaborative filtering, as described in Chapter 9 of Mining of Massive Datasets (Leskovec et al., 2020, Chapter 9), addresses the recommendation problem by constructing a matrix of user-item ratings and computing the cosine similarity between item vectors to identifying movies that attract similar people. The main scalability challenge is that computing pairwise similarity across all items is an $O(n^2)$ operation. This becomes almost impossible at the scale of this dataset. MinHash, described in Chapter 3 of Mining of Massive Datasets (Leskovec et al., 2020, Chapter 3) and originally by Broder (1997), addresses this issue by turning each item's set of users into a signature using random hash functions, taking advantage of the mathematical property that the probability two signatures agree on any given value equals the Jaccard similarity of their original user sets. Locality Sensitive Hashing banding is then used to group items with any matching signature bands into candidate buckets, ensuring the expensive cosine similarity computation is only on candidate pairs; pairs within the same candidate bucket. This hugely reduces the number of comparisons required while preserving the approximate nearest-neighbour relationships and therefore the recommendation accuracy.

Motivation

The motivation for this project came from an interest in how approximation algorithms and hashing techniques can make large scale recommendation systems more practical. I found MinHash interesting because it provides a way to estimate the similarity without directly comparing every pair of items. After first encountering these ideas in lectures, I remembered reading about the Netflix Prize data science competition and became a lot more interested in the practical challenges behind large scale systems. Reading

Takeuchi's (2015) blog post [The Netflix Prize and Production Machine Learning Systems: An Insider Look](#), further increased my interest in the trade off between speed and accuracy. This project gave me the opportunity to implement MinHash, Locality Sensitive Hashing and collaborative filtering all from scratch on a realistic dataset.

Research Question or Hypothesis

The research question guiding this project is: "What is the difference in movie and tv recommendation accuracy and computational runtime between Locality Sensitive Hashing accelerated collaborative filtering and brute-force collaborative filtering when applied to the Amazon Movies and Television Reviews dataset?" This is a comparative quantitative question directly relevant to the dataset due to the sheer scale; over one million ratings across tens of thousands of items. This results in brute-force pairwise similarity computation being very impractical, therefore establishing a clear need for an approximation based approach. The question is structured around two measurable variables, runtime and accuracy, which together capture the fundamental tradeoff that Locality Sensitive Hashing introduces.

The combined MinHash and Locality Sensitive Hashing pipeline directly addresses the runtime variable of the research question by providing a concrete measurement of the computational cost relative to an extrapolated brute-force time. The accuracy variable is evaluated on a held-out test set using Precision at 10 across three signature lengths of 50, 100 and 200 hash functions respectively. In addition to this, a direct accuracy comparison on a random 2,000-item subset and averaged across 20 randomly sampled 2,000-item subsets was conducted.

Experimental Design and Methods

The implementation followed a five stage pipeline executed in Python using Dask, applied to the Amazon Movies and Television Reviews five-core dataset. The raw gzip-compressed JSON file was loaded into a Dask Bag and parsed in parallel using the `.map()` operation to extract three fields per record: user identifier, item identifier and rating. The parsed data was then converted to a Dask Dataframe and split 80/20 into training and test sets by randomly holding out 20% of each user's ratings, ensuring every test user had training history. Item-to-user sets were constructed from the training data by grouping all users who rated each item three stars or above, retaining only items with a minimum of five positive ratings to ensure enough density in items for reliable similarity estimation. MinHash signatures were then computed from scratch for each item using numpy. The hash functions are of the form $h(x) = (ax + b) \bmod p$, where a and b are randomly generated parameters and p is a large prime. These functions form a universal hash family as described by Carter and Wegman (1979), ensuring independence between hash outputs and the distribution of the user identifiers. For each hash function, the minimum hash value across all users in the set is then retained

as part of the item's MinHash signature. This signature generation step was parallelised across the partitions using Dask Bag *.map()*. Locality Sensitive Hashing banding was then applied to group items into candidate buckets, with the rows per band parameter fixed at two, across all configurations to ensure a fair comparison between 50, 100 and 200 hash functions. Cosine similarity was computed within candidate pairs to build item-item similarity indexes and top-ten recommendations were generated for each test user using a similarity-weighted average of their existing ratings.

Three configurations of 50, 100 and 200 hash functions were used to measure the effect of signature length on both accuracy and runtime. Evaluation was performed on a sample of 500 test users using Precision at 10 against held-out ratings as the primary accuracy metric. Root Mean Squared Error and Mean Absolute Error were also computed where recommendations overlapped predictions to get statistically reliable estimates, however these estimates are not reported as primary results as in some places lacked confidence. Brute-force runtime was extrapolated from a timed 1000 item sample to estimate how long it would take if used on the full dataset. In addition to this a direct accuracy comparison was conducted across both a fixed and random subset, as described above. The following functions were implemented within the pipeline:

- **make_hash_params**: generates random hash function parameters.
- **minhash_signature**: computes item signatures using numpy broadcasting.
- **run_minhash**: parallelises signature computation via Dask Bag.
- **run_lsh_banding**: groups items into candidate buckets.
- **run_similarity**: computes cosine similarity on candidate pairs.
- **generate_recommendations**: produces weighted average collaborative filtering predictions.
- **run_evaluation**: computes Precision at 10 and runtime.
- **run_scalability_test**: measures pipeline runtime across four problem sizes.

Results

Single Processor scalability testing showed that runtime increased almost linearly as dataset size grew across all three hash-function configurations as shown in Figure 2, below.

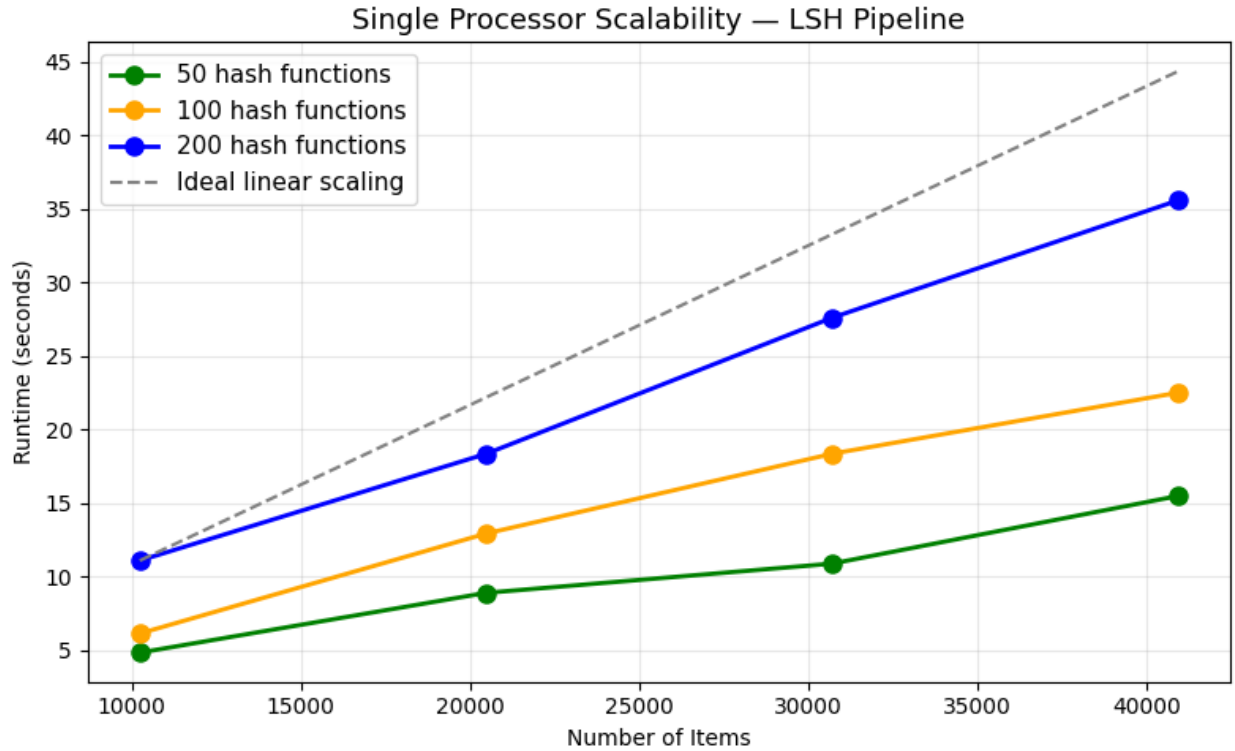


Figure 2: Single processor scalability of the LSH pipeline across three hash function configurations.

However the increase was slightly below the ideal linear scaling line, indicating that my implementation scales reasonably well on a single processor. At 200 hash-functions, processing the full 40,960-item dataset required approximately 36 seconds, while the 50 hash function configuration completed the same task in just over 15 seconds. The consistent separation between the three configurations shows that signature length was a major determinant of runtime, with larger numbers of hash functions proportionally increasing runtime.

Table 1 — Full Pipeline Results

Method	Candidates	Precision@10	Runtime
Brute-force CF (extrapolated)	N/A	0.001	~6 hours 32 mins
LSH CF (50 hash functions)	415,562	0.001	10.7 seconds
LSH CF (100 hash functions)	1,154,443	0.001	31.8 seconds
LSH CF (200 hash functions)	1,535,428	0.002	41.9 seconds

* Brute-force accuracy computed on 2,000-item subset. LSH accuracy computed on full dataset.

Table 1 shows the full pipeline results across all three configurations. The Locality Sensitive Hashing pipeline achieved a speedup of up to 2,197 times over brute-force cosine similarity at 50 hash functions, reducing an estimated six and a half hours of

computation to 10.7 seconds; an excellent result. Precision at 10 remained consistent across all three configurations at between 0.001 and 0.002, indicating that recommendation was preserved regardless of the signature length. It should be noted that the brute-force Precision at 10 figure was from a random 2000-item subset rather than the full dataset. The number of candidate pairs generated increased substantially with hash functions; from 415,562 at 50 hashes to 1,535,428 at 200 hashes. This suggests that larger signatures produce more candidate matches during LSH banding, increasing the number of comparisons required.

Table 2 — Direct Comparison (Random 2,000-item Subset)

Method	Precision@10	Runtime
Brute-force CF	0.001	40.7 seconds
LSH CF (200 hash functions)	0.006	3.3 seconds
Difference	~6x	~12x

Table 2 shows the direct comparison on the identical 2,000-item subset. The Locality Sensitive Hashing approach achieved a Precision at 10 of 0.006 compared to 0.001 for brute-force, while completing the equivalent computation in 3.3 seconds compared to 40.7 seconds, a speedup of over 12 times on identical data. The slightly higher Precision at 10 observed for LSH may reflect the approximation candidate filtering process removing weaker similarity matches, although the sparsity of the dataset means this result should be interpreted cautiously.

Table 3 — Direct Comparison (Average of 20 Random Subsets)

Method	Precision@10	Runtime
Brute-force CF	0.002 (± 0.0005)	17.5 seconds
LSH CF (200 hash functions)	0.007 (± 0.0013)	3.7 seconds
Difference	~3x	~5x

Table 3 presents results averaged across 20 randomly sampled 2000-item subsets. This confirmed the result generalises beyond a single subset; Locality Sensitive Hashing consistently achieved a mean precision at 10 of 0.007 compared to 0.002 for brute-force, higher recommendation accuracy with a fivefold runtime reduction. Together these results directly answer the research question: Locality Sensitive Hashing accelerated collaborative filtering achieves superior recommendation accuracy to brute-force collaborative filtering on controlled evaluation data, at a fraction of the computational cost.

Absolute precision at 10 values were low across all configurations, however this is consistent with known behaviour of collaborative filtering on sparse e-commerce datasets (Su & Khoshgoftaar, 2009). The dataset's long-tail distribution results in small test sets per user, reducing the probability of any recommendation coinciding with a held-out rating. Root Mean Square Error and Mean Absolute Error were computed but found to be statistically unreliable due to the sparse overlap between recommendations and test items; as few as six overlapping predictions across 500 users in some configurations. To try to minimise the effect of this sparsity I did another evaluation that filtered to users and items with a minimum of 20 and then 10 interactions. This evaluation produced a marginal improvement in Precision at 10, confirming dataset sparsity rather than algorithmic limitations as the primary constraint on accuracy.

I also conducted weak scalability testing on Google Cloud Dataproc, shown in the Figure 3, below.

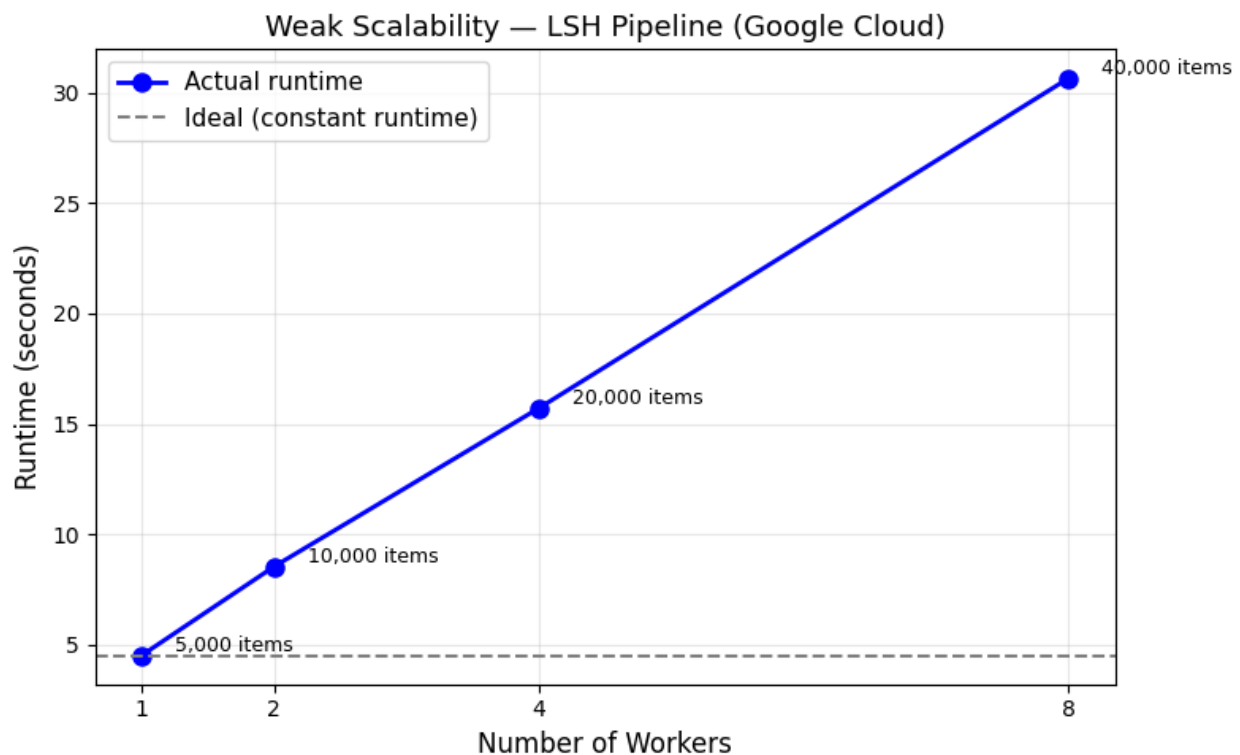


Figure 3: Weak scalability of the LSH pipeline on Google Cloud Dataproc.

This result was almost linear rather than constant runtime growth as workers and problem size doubled; this is consistent with Amdahl's Law (Amdahl, 1967), as the sequential Locality Sensitive Hashing banding step limits parallel efficiency regardless of worker count. I attempted to parallelise the Locality Sensitive Hashing banding step using Dask Bags. While this produced correct candidate pair counts, testing on Google Cloud Dataproc revealed that Dask scheduling overhead seemed to outweigh the parallelisation benefit at this scale.

Conclusion

The research question asked what difference exists in recommendation accuracy and computational runtime between Locality Sensitive Hashing accelerated collaborative filtering and brute-force collaborative filtering. The results show clearly that the accelerated pipeline achieved better recommendation accuracy to brute-force on controlled evaluation data, while reducing computational cost by up to an estimated 2197 times on the full data set. The direct subset comparison confirmed a six times improvement in Precision at 10 on a random subset and this was further validated across 20 randomly sampled subsets, resulting in an average Precision at 10 of 0.007 for Locality Sensitive Hashing compared to 0.002 for brute-force. Together, these results suggest that LSH preserved recommendation quality while substantially reducing runtime.

The implications of these results are significant for any business using recommendation infrastructure at scale without huge engineering resources. The results demonstrate that Locality Sensitive Hashing makes high-quality collaborative filtering practical in environments where brute-force computation isn't. Finding that dataset sparsity being the primary constraint over algorithm design on absolute accuracy also has practical implications. This suggests that investment in data density, through things like increased user engagement may result in more accuracy improvements than improvements to the algorithm itself.

Future work on this project could take several directions. Parallelising the Locality Sensitive Hashing banding step would address the sequential bottleneck identified in weak scalability testing. Although using Dask Bags did not improve the bottleneck, I think with further research and testing it would be possible to completely parallelise. Testing on denser or more heavily filtered data would better isolate algorithm performance from sparsity. Finally, extending the pipeline to incorporate distributed cosine similarity computation on Google Cloud Dataproc would enable evaluation at a scale a lot closer to production recommendation systems, which would therefore provide a more representative assessment of real-world viability.

Critique of Design and Project

The most significant design limitation I encountered was the sensitivity of the Locality Sensitive Hashing banding parameters to dataset sparsity. The initial configuration I had, of 20 bands by 5 rows was selected based on the theoretical guidance from the Mining of Massive Datasets textbook (Leskovec et al., 2020), but this produced only 476 candidate pairs; far too few for meaningful recommendation generation. I later found this occurred because the sparse nature of the dataset meant very few items shared

enough overlapping hash values to match on five consecutive rows within any band, causing the banding to act as a filter. The fix; adjusting to 50 bands by 2 rows and lowering the positive rating threshold from four to three stars, was arrived at through trial and error rather than careful parameter tuning.

A better approach would have involved estimating the expected Jaccard similarity distribution of the dataset before choosing the parameters, therefore helping determine the theoretical optimal band and row values for the level of sparsity. Alternatively to this, a banding strategy that tunes the parameters based on the number of candidate pairs generated could have removed the need for manual adjusting. The weak scalability result also revealed another design limitation; the somewhat sequential nature of the Locality Sensitive Hashing banding step limits how parallel the algorithm can be. This was proven by the linear growth in runtime rather than the flat curve that ideal weak scalability would produce. A possible alternative could be a more distributed approach, such as distributed hash partitioning using a MapReduce style algorithm (Dean & Ghemawat, 2004). This would likely result in more parallelisation and therefore a better weak scalability result.

Reflection

Several course concepts and tools were directly applicable throughout this project. Dask Bags and its `.map()`, `.filter()`, and `.flatten()` operations were essential for my data loading and signature generation pipeline, enabling parallel processing of the dataset that would be impractical to handle using normal Python. The MinHash and Locality Sensitive Hashing algorithms from Chapter 3 of Mining of Massive Datasets and item-based collaborative filtering from Chapter 9, provided the theoretical foundation for the entire project. Google Cloud Dataproc, introduced in the lab assignments helped hugely with running distributed weak scalability tests across multiple workers and problem sizes. NumPy broadcasting was also central to the MinHash implementation, allowing all hash functions to be applied simultaneously across all users in a single matrix operation rather than sequentially.

The project demonstrated that theoretical algorithmic knowledge and practical distributed development skills are vastly linked. Being able to understand the mathematics behind MinHash was essential to diagnosing the Locality Sensitive Hashing parameter sensitivity issue that I encountered early in development. The most valuable lesson was that real world dataset characteristics; especially sparsity, can have a huge influence on results, even more than the algorithmic choices themselves in some cases. Evaluating a system honestly requires understanding and dealing with these factors rather than simply optimising metrics. My attempt at parallelising the banding step further reinforced this. Empirical testing often shows that improvements that look theoretically good on paper do not always translate to gains at scale.

References

- Broder, A. Z. (1997). On the resemblance and containment of documents. *Proceedings of the Compression and Complexity of Sequences*, 21–29.
- Carter, J. L., & Wegman, M. N. (1979). Universal classes of hash functions. *Journal of Computer and System Sciences*, 18(2), 143–154. [https://doi.org/10.1016/0022-0000\(79\)90044-8](https://doi.org/10.1016/0022-0000(79)90044-8)
- Dean, J., & Ghemawat, S. (2004). MapReduce: Simplified data processing on large clusters. *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI)*, 137–150.
- Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). *Mining of massive datasets* (3rd ed.). Cambridge University Press. <http://www.mmms.org>
- McAuley, J. (2023). *Recommender systems and personalization datasets*. UC San Diego. <https://cseweb.ucsd.edu/~jmcauley/datasets.html>
- Su, X., & Khoshgoftaar, T. M. (2009). A survey of collaborative filtering techniques. *Advances in Artificial Intelligence*, 2009, Article 421425. <https://doi.org/10.1155/2009/421425>
- Takeuchi, K. (2015, April 22). The Netflix prize and production machine learning systems: An insider look. *Loren on the Art of MATLAB*. <https://blogs.mathworks.com/loren/2015/04/22/the-netflix-prize-and-production-machine-learning-systems-an-insider-look/>
- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*, 483–485. <https://doi.org/10.1145/1465482.1465560>